

VEMO Guide

How to use the framework, and how it's built — governance for AI coding agents that **accelerates developers**: mechanism over prose, ceremony that scales to risk *and* model strength.

Start **vemo init --preset <stack>**

See state **vemo status**

Learn **vemo explain <topic>**

License **MIT**

0 What VEMO is, in 5 ideas

Drop it into any repo. The rules that matter become **mechanical** (hooks/CI), the advice stays in slim specs loaded just-in-time, and how much process you pay scales to the change's blast radius and your model's strength.

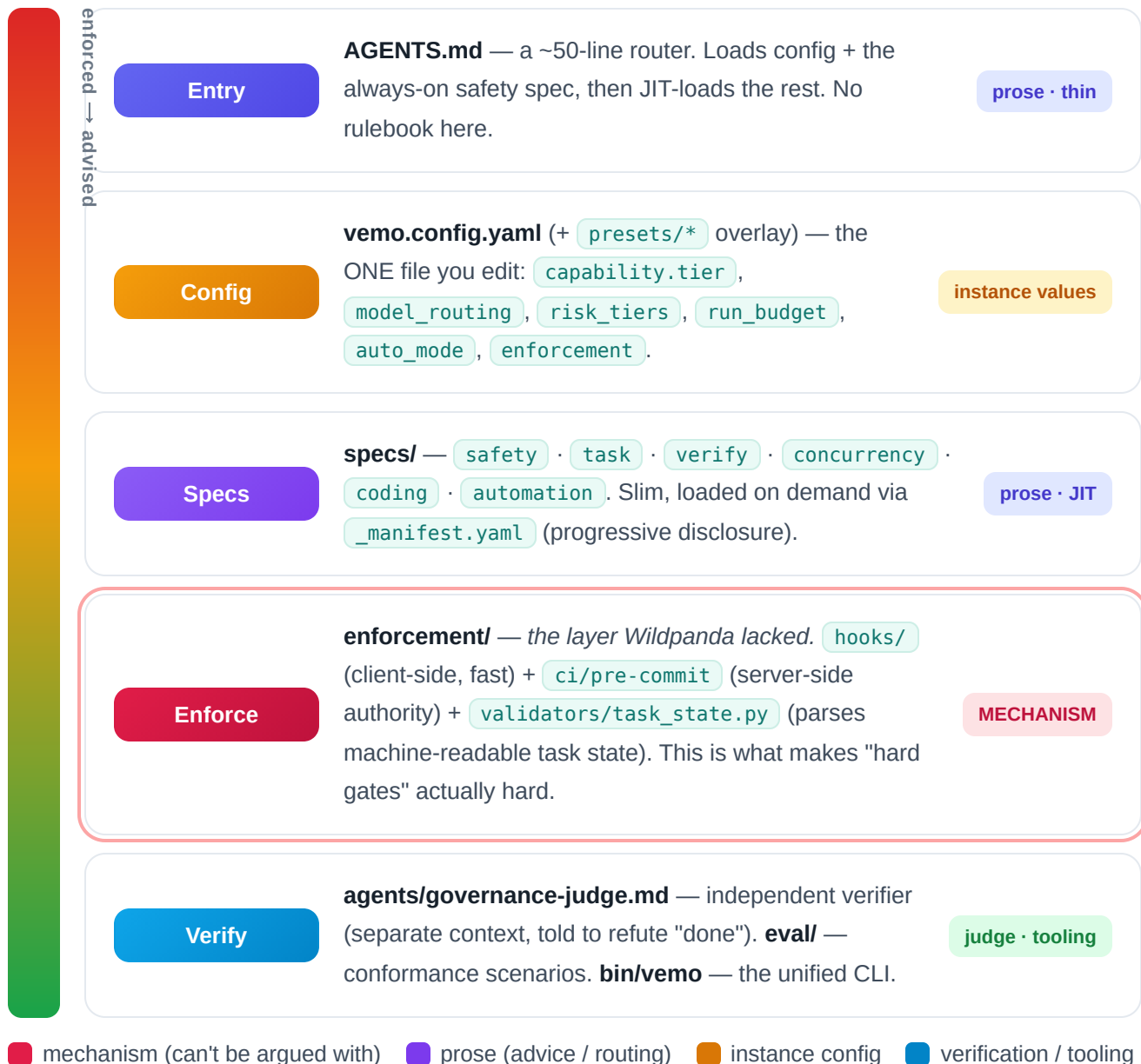
1. **Mechanism over prose** — scope, destructive-cmd, secrets, verify-before-push are enforced by code, not goodwill.
2. **Ceremony scales two ways** — by **risk** (R0 → R2) and by **model** (frontier → low).
3. **The repo is the memory** — durable state in `tasks/<id>.md` front-matter (machine-readable).
4. **Verify, don't trust** — an independent judge flips high-risk gates, not the worker's self-report.
5. **Bounded autonomy** — unattended runs get stop rules + a full decision record.

The one-line mental model

VEMO = a thin router + slim specs + a mechanical enforcement layer + an independent judge. Everything else (risk tiers, capability tiers, auto mode, run budgets) is just *how hard each of those leans in* for a given change and model.

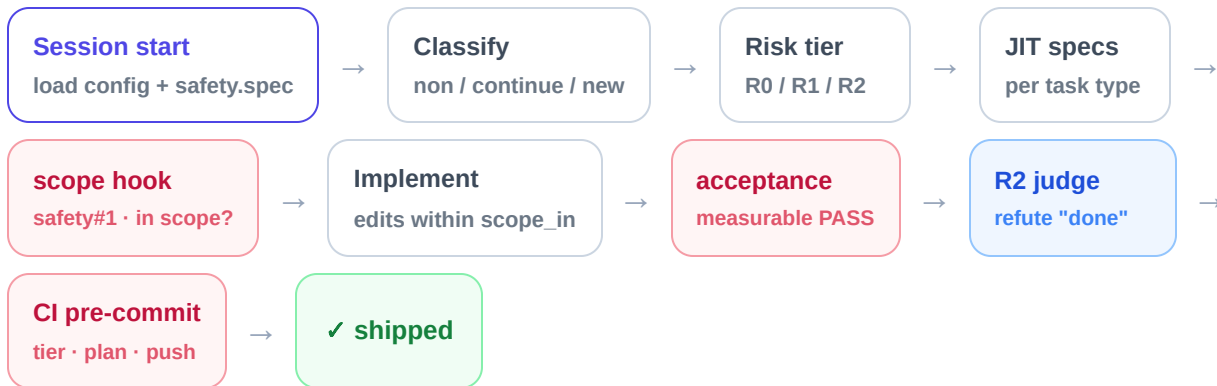
1 Architecture — the layered stack

Five layers, top = what the agent sees first, bottom = what actually guarantees behavior. The left rail shows the gradient that defines VEMO: the lower the layer, the more it is **enforced mechanism** rather than **advisory prose**.



2 Lifecycle — how a change flows through VEMO

A request travels left → right. Plain nodes are steps; **red gates** are mechanical checkpoints (hooks/CI); the **blue gate** is the independent judge. Anything that fails a red gate is blocked, not negotiated.



WHERE AUTONOMY FITS

In **auto mode** the human-approval pauses (R2 plan review, push confirmation, ...) are replaced by **auto-decide + record** — but every red/blue gate above still fires unchanged, and a **run-budget** gate (not shown) hard-stops a runaway when no human is watching.

3 Enforcement — defense in depth

Two walls, so a gate never depends on the agent's goodwill. The client-side hook gives fast feedback inside the agent loop; the server-side CI is the authority that `--no-verify` can't slip past.



WHY TWO WALLS

The hook is fast but client-side (could be absent on a hookless agent). CI is the **authoritative backstop** — it re-runs server-side, so even a human or a misconfigured agent is caught. Belt and suspenders.

READS MACHINE-READABLE STATE

Both walls call `validators/task_state.py`, which parses the task file's YAML front-matter (state · `scope_in` · acceptance · judge verdict). Deterministic — no guessing from prose.

4 The two ceremony axes (visualized)

VEMO's core trick for developer speed: ceremony is not constant. It drops as a change gets **safer** (risk tier ↓) and as the **model gets stronger** (capability tier →). The lightest cell — a trivial change on a frontier model — is almost frictionless; the heaviest — critical change on a weak model — gets the full treatment.

	lowweak model	medium	highOpus 4.8	frontierFable 5
R2 critical	review+judge small steps	review+judge	review+judge	judge+budget self-plan
R1 standard	plan+acc +judge	plan+acc judge(crit)	plan+acc self-verify	self-test +budget
R0 trivial	plan-inline	plan-inline	inline	inline

lightest
moderate
heaviest

Set the model axis once in `capability.tier`; the risk axis is auto-derived from the changed paths (and CI blocks a self-downgrade). Raise `tier` when you upgrade your model and the whole grid lightens automatically. The independent judge stays on R2 across all tiers — autonomy raises the value of verification, never lowers it.

5 Usage

Start without thinking (presets + one command); think when you need to (everything is a documented knob).

Install & initialize

```
# 1) drop VEMO into your repo
cp -r VEMO/{AGENTS.md,vemo.config.yaml,specs,enforcement,agents,tasks,bin,presets} your

# 2) one command: installs hooks + git pre-commit, preconfigures build/test for your st
python3 bin/vemo init --preset python          # or: node | cpp | docs  (--dry-run to prev
# 3) add bin to PATH so you can just type `vemo`
export PATH="$PWD/bin:$PATH"
```

Do work

```
# start a task; set its scope. Out-of-scope edits are then blocked automatically.
cp tasks/_TASK_TEMPLATE.md tasks/T-myfeature.md
# edit front-matter:  scope_in: ["src/feature/**"]  risk: R1
vemo status          # tier / enforcement / budget / auto / active task
vemo explain gates   # what's enforced vs advised
```

Modes (turn on only what you need)

Mode	What it gives you	How
Risk tiers	Less process for low-risk work (R0 fast path)	auto-derived from paths; set globs in <code>risk_tiers</code>
Capability tier	Less hand-holding as the model gets stronger	<code>capability.tier:</code> <code>frontier high medium low</code>
Auto mode	Unattended runs: decide + record instead of asking	<code>vemo auto on</code> (OFF by default, TTL-bounded)
Run budget	Stop rules so a long run can't "run until cut off"	<code>run_budget</code> in config; hard-stops only when unattended

WHEN A GATE BLOCKS YOU

It's not negotiable — fix the cause, don't retry verbatim or `--no-verify` around it (CI re-checks). Out of scope? Either add the file to the task's `scope_in` via a replan, or it belongs to a different task.

6 The `vemo` CLI

One verb-based entry point. Run `vemo` with no args for the map, or `vemo explain <topic>` to learn a concept in one sentence.

Command	Does
<code>vemo init [--preset python node cpp docs] [--dry-run]</code>	Set up VEMO in this repo: apply a stack preset, create <code>tasks/</code> , install hooks + git pre-commit.
<code>vemo status</code>	Plain-language dashboard: version, capability tier, enforcement on/off, run budget, auto mode, active task.
<code>vemo doctor</code>	Health check: config + task front-matter sanity, hooks installed, tools present.
<code>vemo explain <topic></code>	One-sentence explainer: <code>tiers · gates · auto · budget · judge · capability · presets</code> .
<code>vemo auto on off status</code>	Full-auto (unattended) mode. <code>on --max-tier R2 --allow-r2</code> to raise the ceiling.
<code>vemo budget status reset</code>	Inspect / reset the run-budget counter (calls <code>· files · wall-clock</code>).
<code>vemo tier <paths...></code>	Show the required risk tier for the given paths (R0/R1/R2).
<code>vemo check <path></code>	Is a path inside the active task's <code>scope_in</code> ? (in-scope / out-of-scope)

7 Where to go next (docs map)

Organized by need (Diátaxis): learn → do → look-up → understand.

I want to...	Doc
Get working in 5 minutes (tutorial)	QUICKSTART.md
Understand the model (explanation)	MENTAL_MODEL.md
Find the right doc fast (index)	INDEX.md
See the design reasoning	ITERATION_LOG.html · VEMO_REVIEW_EN.html
See the Fable 5 / Mythos-driven changes	FABLE5_MYTHOS_OPTIMIZATION.md
Look up config / CLI / changes	vemo.config.yaml · vemo help · CHANGELOG.md